ASSIGNMENT-11.5

Task 1: Stack Implementation

Prompt:

Implement a stack using a list with push, pop, peek, and is_empty methods.

Code:

Assignment 11.5 > Task1.py > Stack > pop

```python
# implement a stack using a list with push, pop, peek, and is_empty methods.
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        else:
            raise IndexError("Pop from an empty stack")

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        else:
            raise IndexError("Peek from an empty stack")

    def is_empty(self):
        return len(self.stack) == 0
# Example usage
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)
print("Top of stack:", stack.peek())  # Output: Top of stack: 3
print("Pop from stack:", stack.pop())  # Output: Pop from stack: 3
print("Top of stack after pop:", stack.peek())  # Output: Top of stack after pop: 2
print("Is stack empty?", stack.is_empty())  # Output: Is stack empty? False
```

Output:

```
Top of stack: 3
Pop from stack: 3
Top of stack after pop: 2
Is stack empty? False
```

Justification:

A stack is a Last-In-First-Out (LIFO) data structure that allows for efficient addition and removal of elements. Using a list to implement a stack is straightforward and provides O(1) time complexity for push and pop operations. The peek method allows us to view the top element without modifying the stack, and the is_empty method helps us check if the stack has any elements before performing operations.

Task 2:

Prompt:

implement a queue using a list with enqueue, dequeue, peek, and size methods.

Code:

```python
# implement a queue using a list with enqueue,dequeue,peek,and size methods.
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            raise IndexError("Dequeue from an empty queue")

    def peek(self):
        if not self.is_empty():
            return self.queue[0]
        else:
            raise IndexError("Peek from an empty queue")

    def size(self):
        return len(self.queue)

    def is_empty(self):
        return len(self.queue) == 0
# Example usage
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)
print("Queue size:", queue.size())  # Output: Queue size: 3
print("Peek:", queue.peek())        # Output: Peek: 1
print("Dequeue:", queue.dequeue())  # Output: Dequeue: 1
print("Queue size after dequeue:", queue.size())  # Output: Queue size after dequeue: 2
```

Output:

```
Queue size: 3
Peek: 1
Dequeue: 1
Queue size after dequeue: 2
```

Justification:

A queue is a First-In-First-Out (FIFO) data structure, and using a list allows us to easily implement the enqueue and dequeue operations. The enqueue operation appends items to the end of the list, while the dequeue operation removes items from the front of the list. The peek method allows us to see the front item without removing it, and the size method provides the current number of items in the queue.

Task 3:

Prompt:

implement a singly linked list with insert and delete methods.

Code:

Assignment 11.5 > Task3.py > SinglyLinkedList > insert

```python
# implement a singly linked list with insert and delete methods.
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def delete(self, key):
        current = self.head
        previous = None
        while current and current.data != key:
            previous = current
            current = current.next
        if current is None:
            return  # Key not found
        if previous is None:
            self.head = current.next  # Deleting the head node
        else:
            previous.next = current.next  # Bypass the node to delete

    def display(self):
        current = self.head
        while current:
            print(current.data, end=' ')
            current = current.next
        print()
# Example usage
linked_list = SinglyLinkedList()
linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
print("Linked List after insertion:")
linked_list.display()
```

Output:

```
Linked List after insertion:
1 2 3
```

Justification:

The insert method adds a new node to the end of the linked list, while the delete method removes a node with a specific key. The display method is used to print the contents of the linked list. This implementation allows for efficient insertion and deletion of nodes in a singly linked list structure.

Task 4:

Prompt:

implement a hash table with basic insert,search,and delete methods.

Code:

Assignment 11.5 > Task4.py > HashTable > insert

```python
# implement a hash table with basic insert,search,and delete methods.
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        if self.table[index] is None:
            self.table[index] = [(key, value)]
        else:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    self.table[index][i] = (key, value)
                    return
            self.table[index].append((key, value))

    def search(self, key):
        index = self.hash_function(key)
        if self.table[index] is not None:
            for k, v in self.table[index]:
                if k == key:
                    return v
        return None

    def delete(self, key):
        index = self.hash_function(key)
        if self.table[index] is not None:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    del self.table[index][i]
                    return True
        return False
# Example usage
hash_table = HashTable(10)
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
hash_table.delete("name")
print(hash_table.search("name"))  # Output: None
```

Output:

```
Alice
30
None
```

Justification:

A hash table is an efficient data structure for storing and retrieving key-value pairs. It provides average-case O(1) time complexity for insertions, deletions, and searches, making it ideal for managing appointment records where quick access is essential.

Task 5:

Prompt:

Implement a graaph using an adjacency list. The graph should support adding vertices, adding edges, and displaying the graph.

Code:

Assignment 11.5 > Task5.py > ...

```python
#Implement a graaph using an adjacency list. The graph should support adding vertices, adding edges, and displaying the graph.
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)

    def display(self):
        for vertex in self.graph:
            print(f"{vertex}: {self.graph[vertex]}")
# Example usage
g = Graph()
g.add_vertex("A")
g.add_vertex("B")
g.add_vertex("C")
g.add_edge("A", "B")
g.add_edge("A", "C")
g.display()
```

Output:

```
A: ['B', 'C']
B: ['A']
C: ['A']
```

Justification:

An adjacency list is an efficient way to represent a graph, especially when the graph is sparse. It allows for easy addition of vertices and edges, and it provides a clear way to display the graph structure. The time complexity for adding vertices and edges is O(1), and displaying the graph takes O(V + E) time, where V is the number of vertices and E is the number of edges.

Task 6:

Prompt:

Select suitable data structures for the Smart Hospital Management System features. Create a table with feature, chosen data structure, and 2–3 sentence justification. Also implement one feature in Python with comments and docstrings.

Code:

```
Assignment 11.5 > Task6.py > PatientRecords > __init__
#Select suitable data structures for the Smart Hospital Management System features. Create a table with feature, chosen data structure, and 2-3 sentence justification. Also implement one feature in Python with comments and docstrings.
# Feature: Patient Records Management
# Chosen Data Structure: Hash Table
# Justification: A hash table allows for efficient insertion, search, and deletion of patient records using unique identifiers (e.g., patient ID). It provides constant average time complexity for these operations, making it suitable for managing a large number of pati
class PatientRecords:
    """
    A class to manage patient records using a hash table.
    """

    def __init__(self):
        """
        Initializes the patient records with an empty hash table.
        """
        self.records = {}

    def add_record(self, patient_id, record):
        """
        Adds a new patient record to the hash table.

        :param patient_id: Unique identifier for the patient
        :param record: A dictionary containing patient information
        """
        self.records[patient_id] = record

    def get_record(self, patient_id):
        """
        Retrieves a patient record from the hash table.

        :param patient_id: Unique identifier for the patient
        :return: The patient record if found, else None
        """
        return self.records.get(patient_id)

    def delete_record(self, patient_id):
        """
        Deletes a patient record from the hash table.

        :param patient_id: Unique identifier for the patient
        :return: True if the record was deleted, else False
        """
        if patient_id in self.records:
            del self.records[patient_id]
            return True
        return False
# Example usage
patient_records = PatientRecords()
patient_records.add_record("12345", {"name": "John Doe", "age": 30, "condition": "Flu"})
print(patient_records.get_record("12345"))  # Output: {'name': 'John Doe
patient_records.delete_record("12345")
print(patient_records.get_record("12345"))  # Output: None
```

Output:

```
{'name': 'John Doe', 'age': 30, 'condition': 'Flu'}
None
```

Justification:

A hash table allows for efficient insertion, search, and deletion of patient records using unique identifiers (e.g., patient ID). It provides constant average time complexity for these operations, making it suitable for managing a large number of patient records.

Task 7:

Prompt:

Select suitable data structures for the Smart City Traffic Control System features. Create a table with feature, chosen data structure, and 2–3 sentence justification. Also implement one feature in Python with comments and docstrings.

Code:

```python
#Select suitable data structures for the Smart City Traffic Control System features. Create a table with feature, chosen data structure, and 2-3 sentence justification. Also implement one feature in Python with com
# Feature: Traffic Signal Management
# Chosen Data Structure: Graph
# Justification: A graph data structure can represent the traffic network, where intersections are nodes and roads are edges. This allows for efficient pathfinding algorithms to optimize traffic flow and manage sig
class TrafficSignalManagement:
    """
    A class to manage traffic signals using a graph data structure.
    """

    def __init__(self):
        """
        Initializes the traffic signal management system with an empty graph.
        """
        self.graph = {}

    def add_intersection(self, intersection):
        """
        Adds a new intersection (node) to the graph.

        :param intersection: The name of the intersection
        """
        if intersection not in self.graph:
            self.graph[intersection] = []

    def add_road(self, intersection1, intersection2):
        """
        Adds a road (edge) between two intersections in the graph.

        :param intersection1: The first intersection
        :param intersection2: The second intersection
        """
        if intersection1 in self.graph and intersection2 in self.graph:
            self.graph[intersection1].append(intersection2)
            self.graph[intersection2].append(intersection1)

    def display(self):
        """
        Displays the traffic signal management graph.
        """
        for intersection in self.graph:
            print(f"{intersection}: {self.graph[intersection]}")
# Example usage
traffic_signal_management = TrafficSignalManagement()
traffic_signal_management.add_intersection("A")
traffic_signal_management.add_intersection("B")
traffic_signal_management.add_intersection("C")
traffic_signal_management.add_road("A", "C")
traffic_signal_management.add_road("A", "B")
traffic_signal_management.add_road("B", "C")
traffic_signal_management.display()
```

Output:

```
A: ['C', 'B']
B: ['A', 'C']
C: ['A', 'B']
```

Justification:

A graph data structure can represent the traffic network, where intersections are nodes and roads are edges. This allows for efficient pathfinding algorithms to optimize traffic flow and manage signal timings based on real-time traffic conditions.

Task 8:

Prompt:

Choose suitable data structures for Smart E-Commerce features, give a table with justification, and implement one feature in Python with comments.

Code:

Assignment 11.5 > Task8.py > ProductCatalog

```python
#Choose suitable data structures for Smart E-Commerce features, give a table with justification, and implement one feature in Python with comments.
# Feature: Product Catalog Management
# Chosen Data Structure: Trie
class ProductCatalog:
    """
    A class to manage a product catalog using a trie data structure.
    """

    def __init__(self):
        """
        Initializes the product catalog with an empty trie.
        """
        self.trie = {}

    def add_product(self, product_name):
        """
        Adds a new product to the trie.

        :param product_name: The name of the product to be added
        """
        current_node = self.trie
        for char in product_name:
            if char not in current_node:
                current_node[char] = {}
            current_node = current_node[char]
        current_node['$'] = True  # Mark the end of a product name

    def search_product(self, prefix):
        """
        Searches for products in the trie that match the given prefix.

        :param prefix: The prefix to search for
        :return: A list of products that match the prefix
        """
        current_node = self.trie
        for char in prefix:
            if char not in current_node:
                return []  # No products found with the given prefix
            current_node = current_node[char]

        return self._collect_products(current_node, prefix)

    def _collect_products(self, node, prefix):
        """
        Helper method to collect all products from a given node in the trie.

        :param node: The current node in the trie
        :param prefix: The prefix accumulated so far
        :return: A list of products collected from the trie
        """
        products = []
        if '$' in node:
            products.append(prefix)  # Found a complete product name

        for char in node:
            if char != '$':
                products.extend(self._collect_products(node[char], prefix + char))

        return products
# Example usage
product_catalog = ProductCatalog()
product_catalog.add_product("Laptop")
product_catalog.add_product("Lamp")
product_catalog.add_product("Phone")
print(product_catalog.search_product("La"))  # Output: ['Laptop', 'Lamp']
print(product_catalog.search_product("Ph"))  # Output: ['Phone']
print(product_catalog.search_product("Ta"))  # Output: []
```

Output:

```
['Laptop', 'Lamp']
['Phone']
[]
```

Justification:

A trie (prefix tree) is an efficient data structure for managing a product catalog, especially when dealing with a large number of products. It allows for fast retrieval of products based on their names or prefixes, making it ideal for search functionalities in an e-commerce platform.